Chapter 3 – Instruction-Level Parallelism and its Exploitation (Part 3)

ILP vs. Parallel Computers

Dynamic Scheduling (Section 3.4, 3.5)

Dynamic Branch Prediction (Section 3.3, 3.9, and Appendix C)

Hardware Speculation and Precise Interrupts (Section 3.6)

Multiple Issue (Section 3.7)

Static Techniques (Section 3.2, Appendix H)

Limitations of ILP

Multithreading (Section 3.11)

Putting it Together (Mini-projects)

Review: Superpipelining

$$CPU_{time} = \frac{instructions}{program} X \frac{cycles}{instruction} X \frac{time}{cycle}$$

Solution: superpipelining for decreasing the cycle time

cycle time = 200ps + 50ps = 250ps

cycle time = 100ps+ 50ps = 150ps

Review: Superpipelining

Are the more pipelines stages the better? **No**

cycle time = 200ps + 50ps = 250ps
latency per ins = 1250ps
50ps / 250ps = 20%

cycle time = 100ps+ 50ps = 150ps
latency per ins = 1500ps
50ps / 150ps = 33%

Review: Changes to the pipelining stages

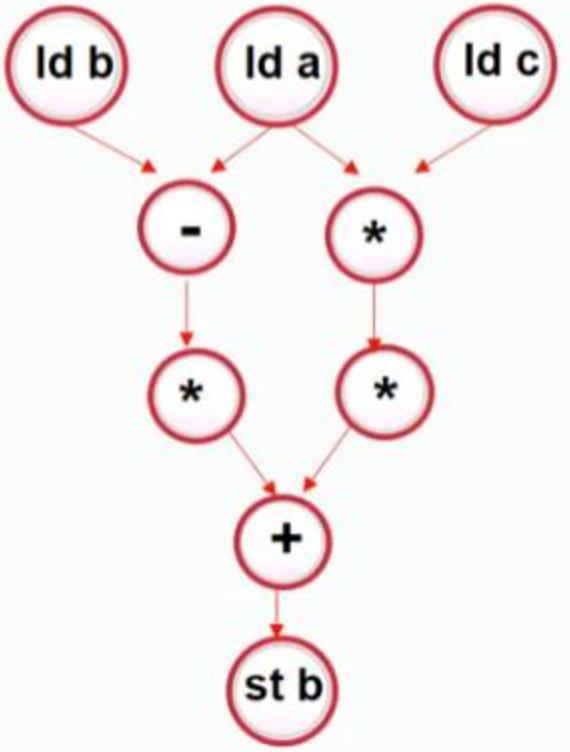
- 1986, MIPS R2000: 5 stages
- 1988, MIPS R3000 5 stages
- 1991, MIPS R4000 (64 bits): 8 stages
- 1997, ARM9: 5 stages
- 2002, ARM11: 8 stages
- 2009, Cortex-A8: 13 stages
- 2011, Cortex-A15: 15 stages
- 2013, Cortex-A57: 15 stages

- 1993, Pentium: 5 stages
- 1995, Pentium Pro: 12 stages
- 2004, Pemtium 4(Prescott): **31 stages**
- 2006, Core 2 Duo(Merom): 14 stages
- 2008, Core i7(Nehalem): 16 stages
- 2013, Core i7(Haswell): 14 stages

ILP

$$D = 3 * (a - b) + 7 * a * c ;$$

ld a
ld b
sub a-b
mul 3(a-b)*
ld c
*mul a*c*
*mul 7*a*c*
*add 3(a-b) + 7*a*c*
st d



Beyond Pipelining (Section 3.7)

Limits on Pipelining

- Latch overheads & signal skew
- Unpipelined instruction issue logic (Flynn limit: CPI $\geq$ 1)

Multiple issue:

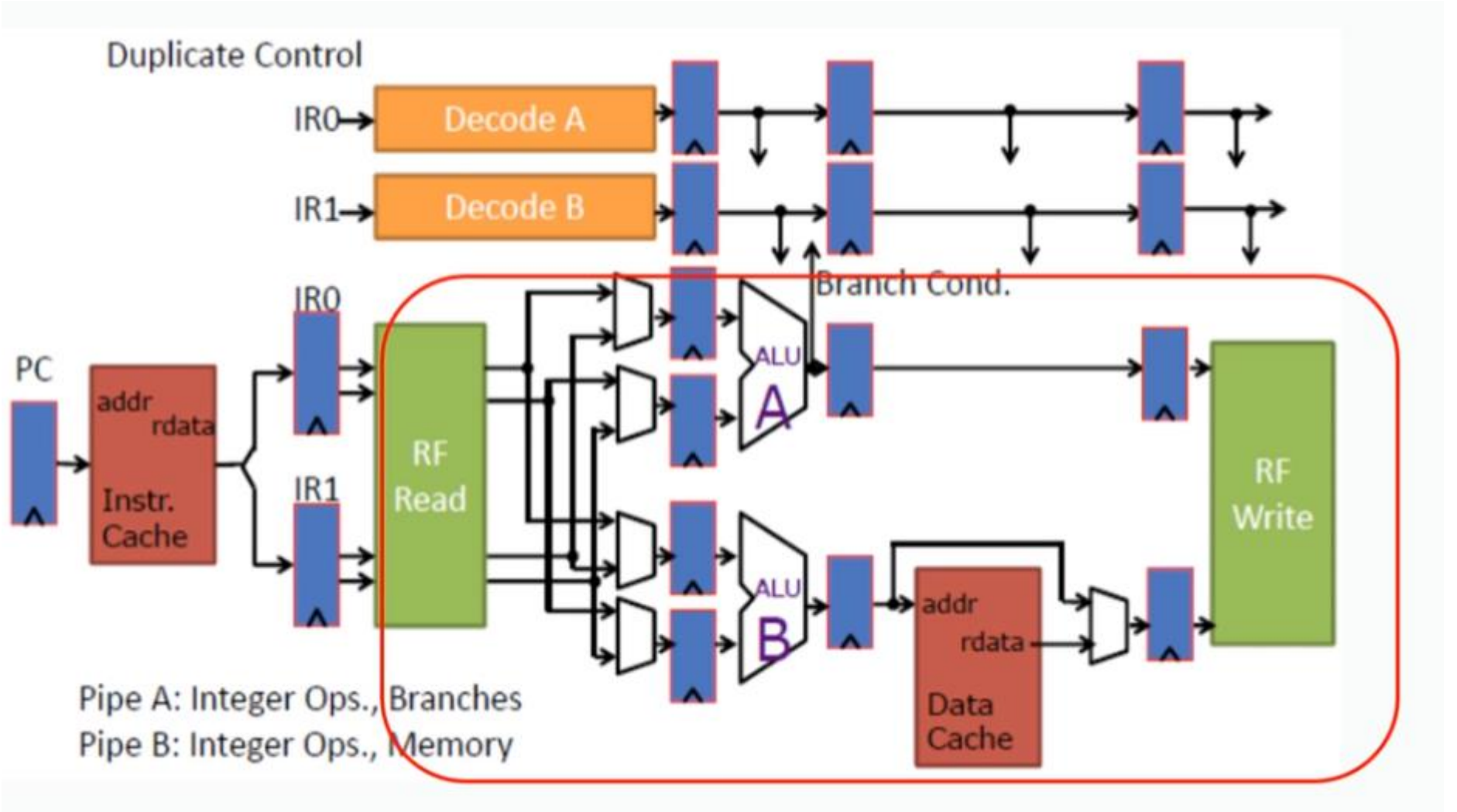
Two techniques for parallelism in instruction issue

- Superscalar
  - Hardware determines which of next *n* instructions can issue in parallel
  - Maybe statically or dynamically scheduled
- VLIW – Very Long Instruction Word
  - Compiler packs multiple independent operations into an instruction

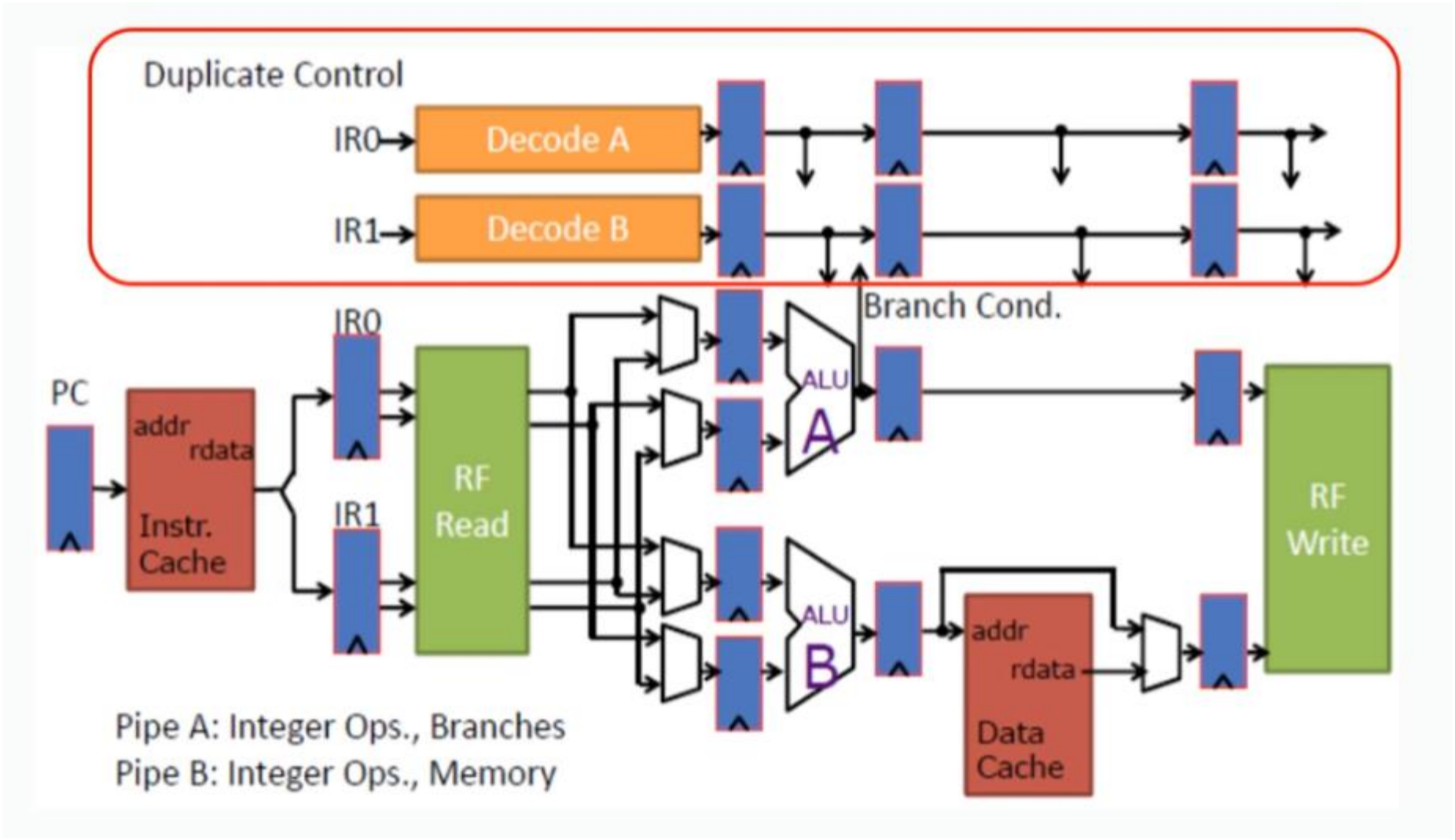
Five Primary Approaches for Multi-Issue Processors

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the ARM Cortex-A8
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Dynamic Two-Issue Superscalar Processor with In-Order



Dynamic Two-Issue Superscalar Processor with In-Order

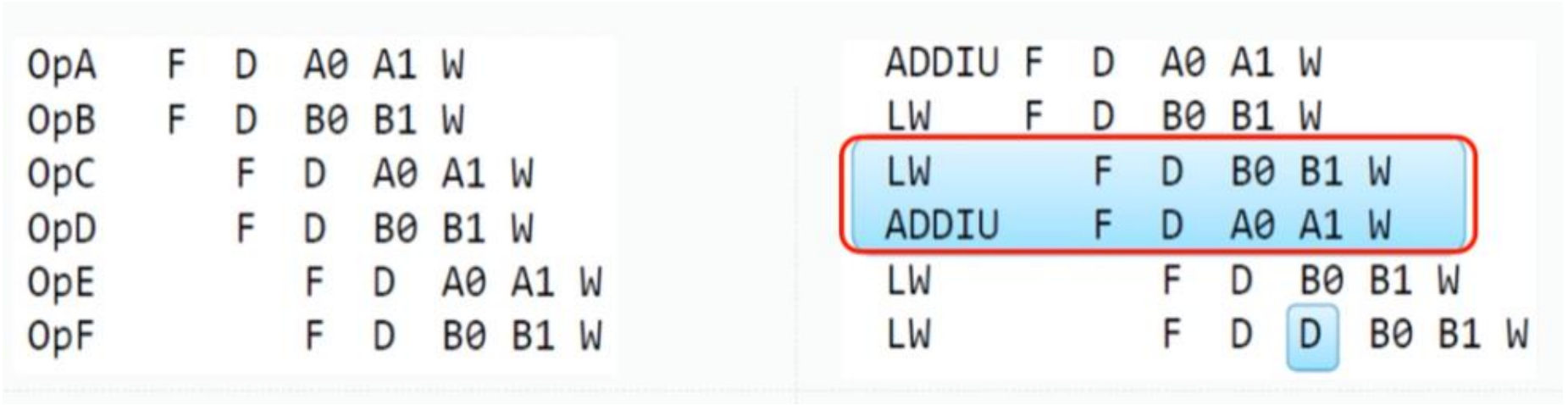


Simple 5-Stage Superscalar Pipeline

	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1	IF	ID	EX	MEM	WB				
i+2		IF	ID	EX	MEM	WB			
i+3		IF	ID	EX	MEM	WB			
i+4			IF	ID	EX	MEM	WB		
i+5			IF	ID	EX	MEM	WB		
i+6				IF	ID	EX	MEM	WB	
i+7				IF	ID	EX	MEM	WB	
i+8					IF	ID	EX	MEM	WB
i+9					IF	ID	EX	MEM	WB

$CPI_{ideal} = 0.5$

Dynamic Two-Issue Superscalar Processor with In-Order

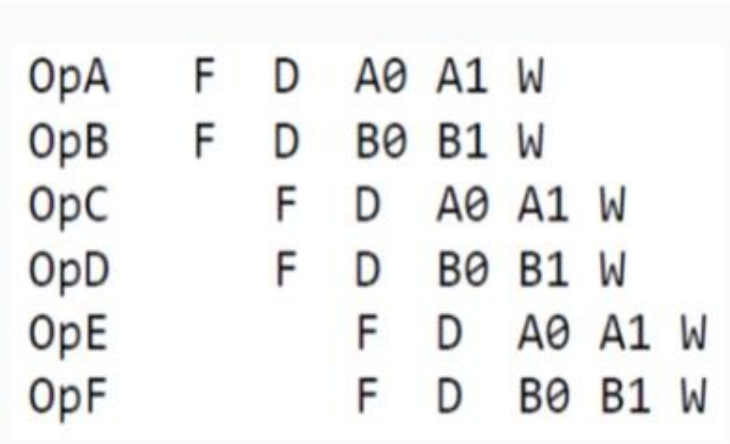


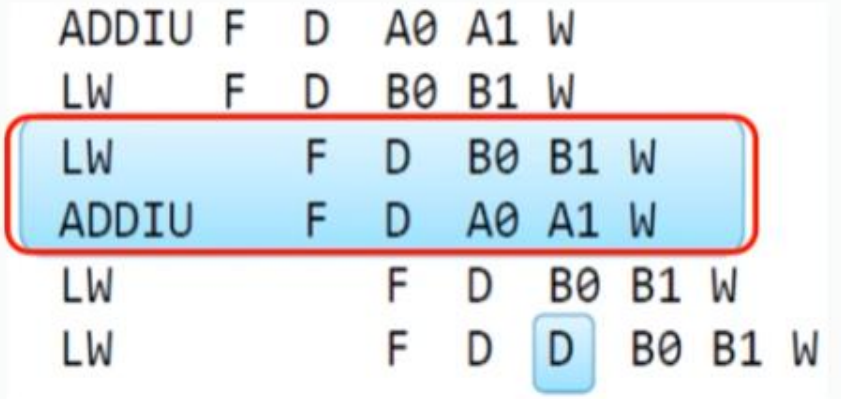
$CPI_{ideal} = 0.5$

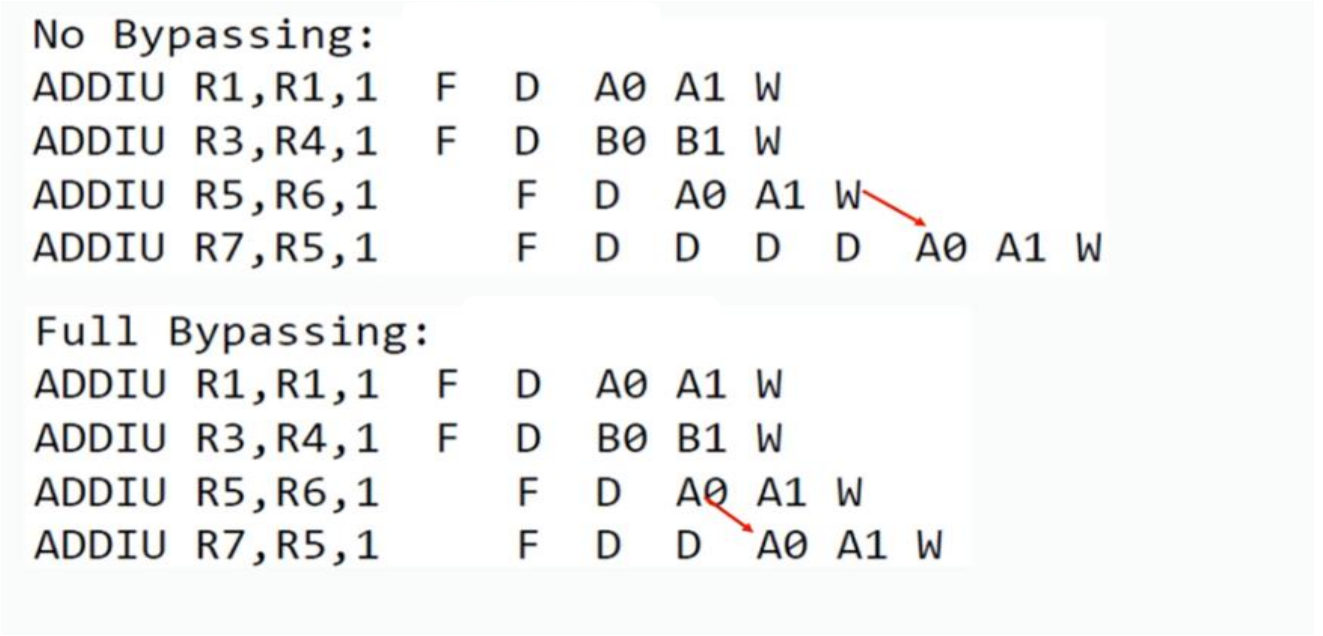
Control Logic:

- **Switch Issue Slots**
- **Detect Hazards**

Dynamic Two-Issue Superscalar Processor with In-Order







$CPI_{ideal} = 0.5$

Control Logic:

- **Switch Issue Slots**
- **Detect Hazards**

Out-of-Order Execution

	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [r1] → r2	F	D	X	M	W								
add r2 + r3 → r4	F	D	d*	d*	X	M	W						
xor r4 ^ r5 → r6		F	D	d*	d*	X	M	W					
ld [r7] → r8		F	D	p*	p*	X	M	W					

In-Order

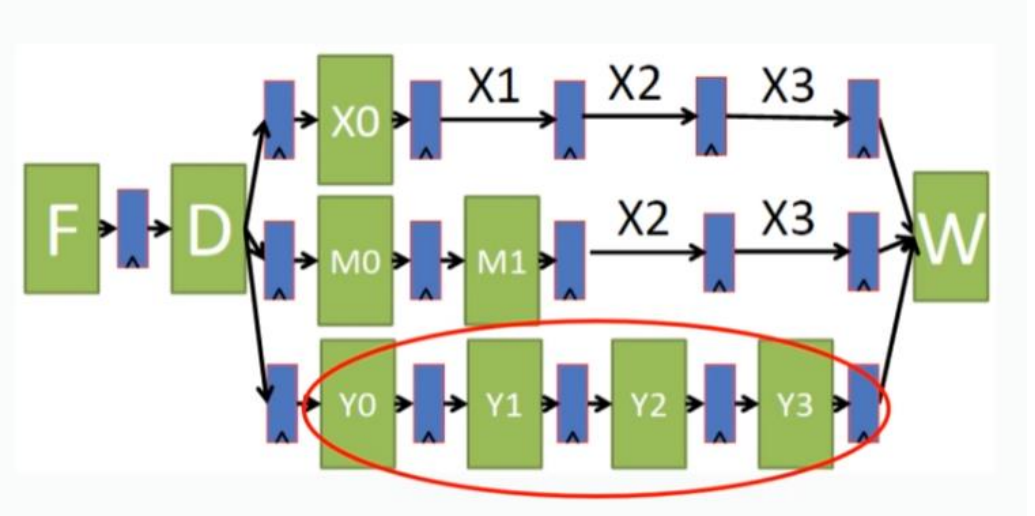
	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [r1] → r2	F	D	X	M	W								
add r2 + r3 → r4	F	D	d*	d*	X	M	W						
xor r4 ^ r5 → r6		F	D	d*	d*	X	M	W					
ld [r7] → r8		F	D	X	M	W							

Out-of-Order

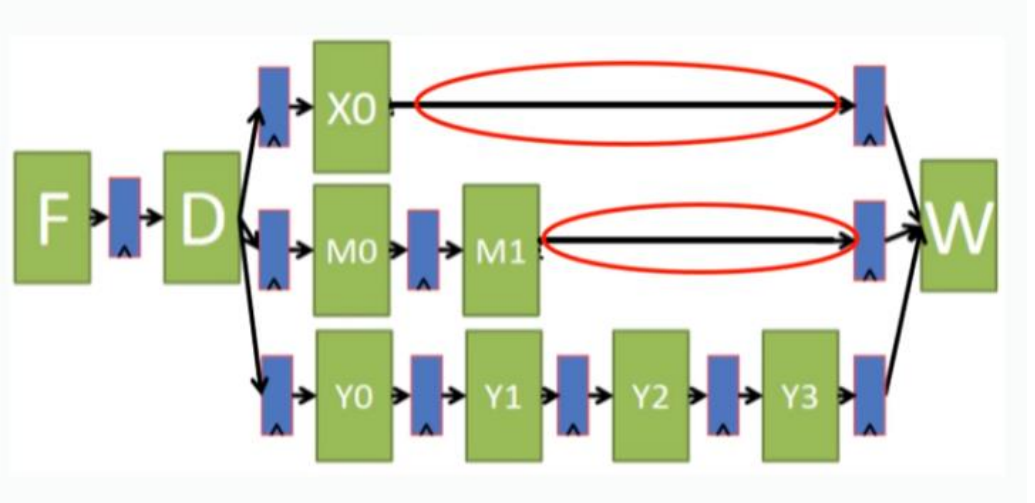
Out-of-Order Execution:

A situation in pipelined execution when an instruction blocked from executing does not cause the following instructions to wait.

Out-of-Order Execution

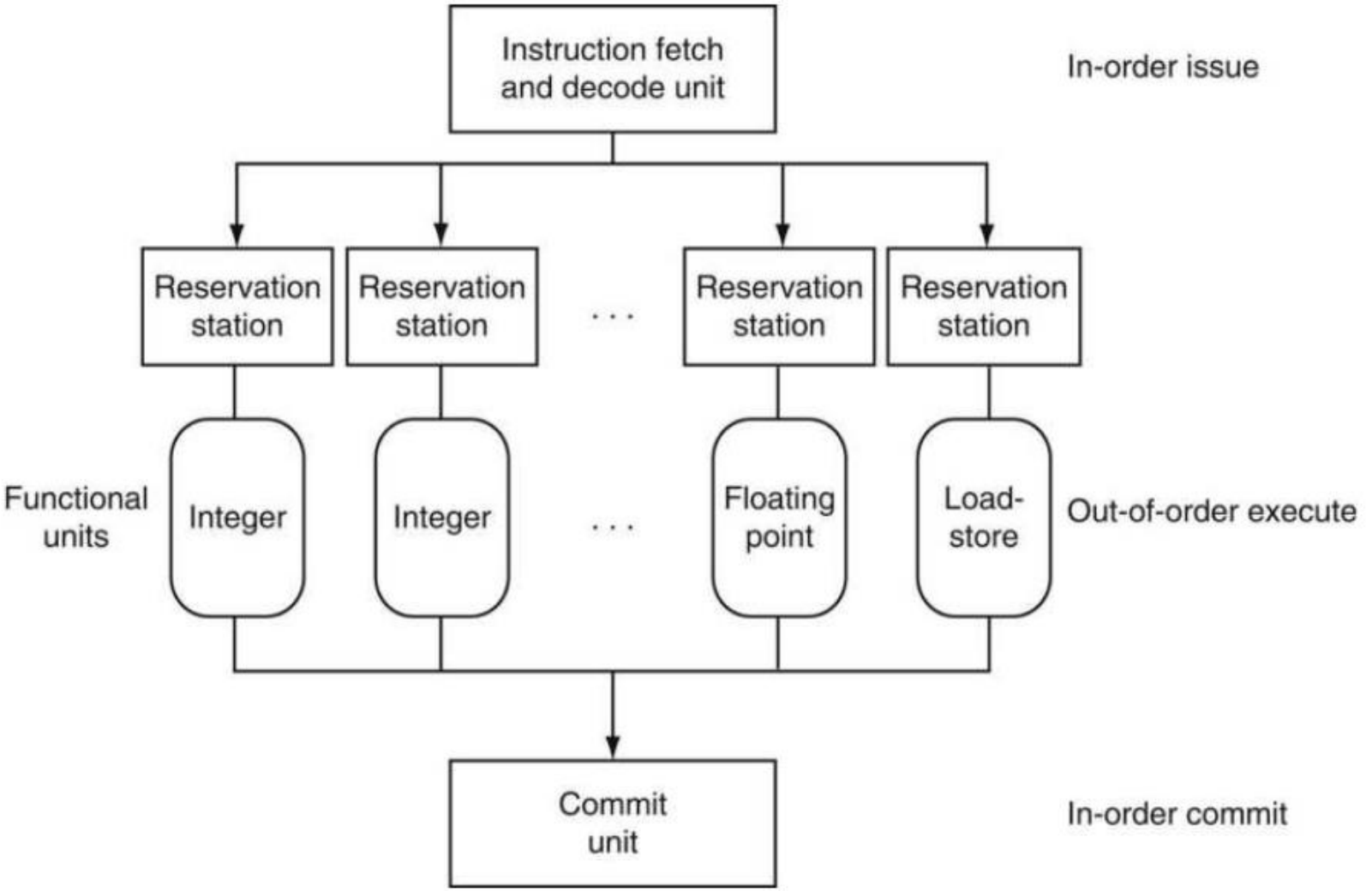


In-Order



Out-of-Order

Dynamic Pipeline Scheduling



Data Hazards due to Out-of-Order Execution

```
LW     $t0,0($s1)
ADDU   $t0,$t1,$s2
SUB    $t2, $t0, $s2
```

Output Dependency:

- **LW is later than ADDU write register t0**
- **SUB gets the wrong result from register t0**

Aka WAW

```
DIVD   F0,F2,F4
ADDD   F10,F0,F8
SUBD   F8,F8,F14
```

Anti Dependency:

- **SUBD writes the register F8 before ADDD reads the register F8**
- **ADDD gets the wrong result from the register F8**

Aka WAR

Register Renaming

Output Dependency:

```
LW     $t0,0($s1)
ADDU   $t0,$t1,$s2
SUB    $t2, $t0, $s2
```

```
LW     $t0a,0($s1)
ADDU   $t0b,$t1,$s2
SUB    $t2, $t0b, $s2
```

Anti Dependency:

```
DIVD   F0,F2,F4
ADDD   F10,F0,F8
SUBD   F8,F8,F14
```

```
DIVD   F0,F2,F4
ADDD   F10,F0,F8a
SUBD   F8b,F8a,F14
```

Superscalar, cont.

IF	Parallel access to I-cache Require alignment?
ID	Replicate logic Fixed-length instructions? HANDLE INTRA-CYCLE HAZARDS
EX	Parallel/pipelined (as before)
MEM	> 1 per cycle? If so, hazards & multi-ported D-cache
WB	Different register files? Multi-ported register files?

Progression:
- Integer + floating-point
- Any two instructions
- Any four instructions
- Any n instructions?

Example Superscalar

Assume two instructions per cycle

- One integer, load/store, or branch
- One floating point

Could require 64-bit alignment and ordering of instruction pair.

I F	I F	F I
I F	F I	F I
OK	NOT OK	NOT OK

Best case

- CPI = 0.5
- But ....

Superscalar (Cont.)

Hazards are a big problem

Loads

- Latency is 1 cycle
- Was 1 instruction
- NOW 3 instructions

Branches

- NOW 3 instructions

Floating point loads and stores

- May cause structural hazards
- Additional ports?
- Additional stalls?

Parallelism required =

*Superscalar (Cont.)***

Hazards are a big problem

Loads

- Latency is 1 cycle
- Was 1 instruction
- NOW 3 instructions

Branches

- NOW 3 instructions

Floating point loads and stores

- May cause structural hazards
- Additional ports?
- Additional stalls?

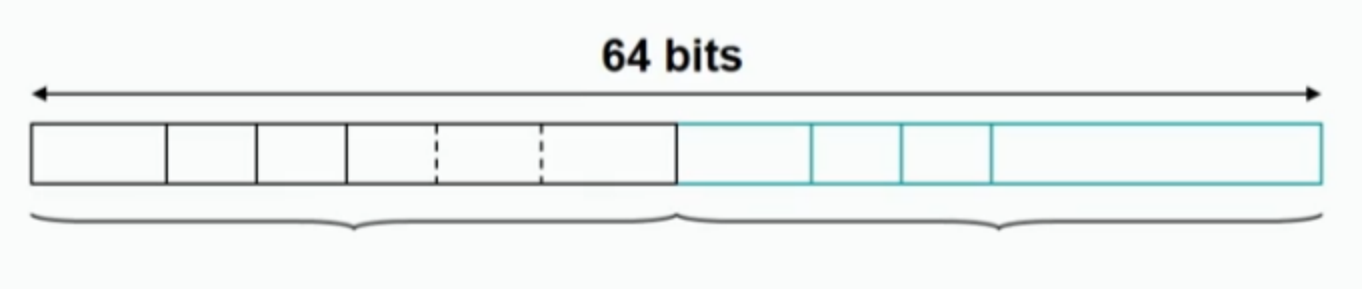
Parallelism required = superscalar degree x operation latency

Static Techniques for ILP - VLIW Processors

VLIW = Very Long Instruction Word Processors

Static multiple issue

- Compiler packs multiple *independent* operations into an instruction
- Like horizontal microcode



*VLIW Processors***

VLIW = Very Long Instruction Word Processors

Static multiple issue

- Compiler packs multiple *independent* operations into an instruction
- Like horizontal microcode

Versus Superscalar

- \+ Issue logic simpler
- \+ Potentially exploit more parallelism

*VLIW Processors***

VLIW = Very Long Instruction Word Processors

Static multiple issue

Compiler packs multiple *independent* operations into an instruction

Like horizontal microcode

Versus Superscalar

+ Issue logic simpler
+ Potentially exploit more parallelism
- Code size explosion
- Complex compiler
- Lock Step
- Binary compatibility difficult across generations

*VLIW Processors***

VLIW = Very Long Instruction Word Processors

Static multiple issue

- Compiler packs multiple *independent* operations into an instruction
- Like horizontal microcode

Versus Superscalar

- \+ Issue logic simpler
- \+ Potentially exploit more parallelism
- \- Code size explosion
- \- Complex compiler
- \- Binary compatibility difficult across generations

Recent VLIWs overcome some problems (e.g., Intel/HP IA-64, TI C6)

Limitations of Multi-Issue Machines

Inherent limitations of ILP

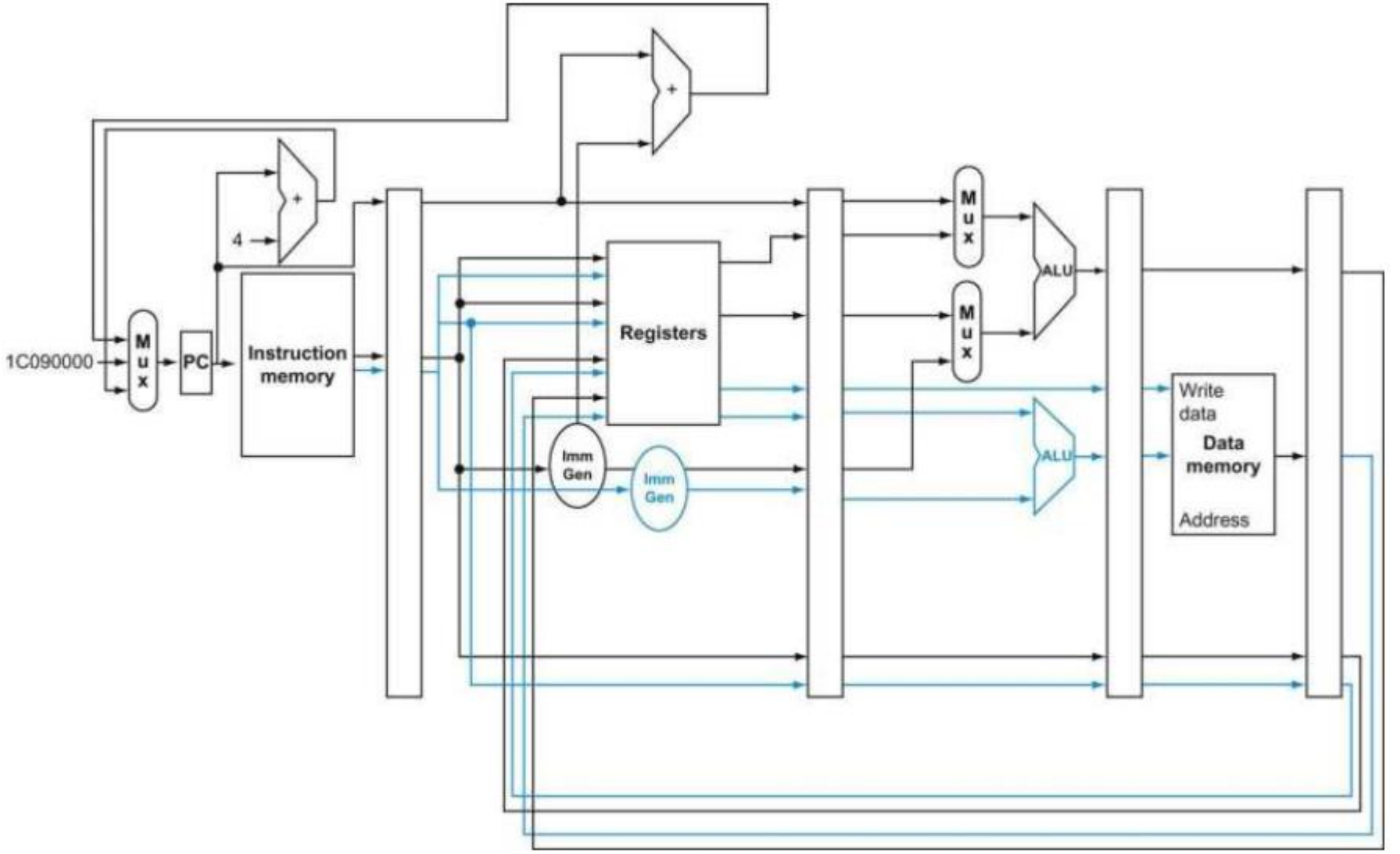
Difficulties in building hardware

- Increase ports to registers
- Increase ports to memory
- Duplicate FUs
- Decoding in superscalar and impact on clock rate

Limitations specific to VLIW

- Code size, binary compatibility

A Static Two-Issue Datapath



A Static Two-Issue Pipeline Example

Add scalar to vector

```
Loop: ld   x31, 0(x20) // x31=array element
      add  x31, x31, x21 // add scalar in x21
      sd   x31, 0(x20) // store result
      addi x20, x20, -8 // decrement pointer
      blt  x22, x20, Loop // compare to loop limit,
                          // branch if    x20 > x22
```

With scheduling

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		ld x31, 0(x20)	1
	addi x20, x20, -8		2
	add x31, x31, x21		3
	blt x22, x20, Loop	sd x31, 8(x20)	4

$CPI_{ideal} = 0.5$, $CPI_{real} = 0.8$

A Static Two-Issue Pipeline Example

Add scalar to vector

```
Loop: ld   x31, 0(x20) // x31=array element
      add  x31, x31, x21 // add scalar in x21
      sd   x31, 0(x20) // store result
      addi x20, x20, -8 // decrement pointer
      blt  x22, x20, Loop // compare to loop limit,
                          // branch if    x20 > x22
```

Loop Unrolling:

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi x20, x20, -32	ld x28, 0(x20)	1
		ld x29, 24(x20)	2
	add x28, x28, x21	ld x30, 16(x20)	3
	add x29, x29, x21	ld x31, 8(x20)	4
	add x30, x30, x21	sd x28, 32(x20)	5
	add x31, x31, x21	sd x29, 24(x20)	6
		sd x30, 16(x20)	7
	blt x22, x20, Loop	sd x31, 8(x20)	8

$CPI_{ideal} = 0.5$, $CPI_{real} = 0.57$

Compiler Techniques to Expose ILP

Many compiler techniques exist

- Several used for multiprocessors as well
- Our focus on techniques specifically for ILP

Loop Unrolling (Section 3.2)

Add scalar to vector

```
Loop: L.D F0, 0(R1)
      stall
      ADD.D F4, F0, F2
      stall
      stall
      S.D 0(R1), F4
      DSUBUI R1, R1, #8
      stall
      BNEZ R1, Loop
      stall
```

With scheduling

```
Loop:  L.D F0, 0(R1)
       DSUBUI R1, R1, #8
       ADD.D F4, F0, F2
       stall
       BNEZ R1, Loop ; Assume delayed branch
       S.D 8(R1), F4
```

$CPI = 2, CPI_{real} = 1.2$

Loop Unrolling

Unrolling the loop

```
Loop:  L.D F0, 0(R1)
       ADD.D F4, F0, F2
       S.D 0(R1), F4
       L.D F6, -8(R1)
       ADD.D F8, F6, F2
       S.D -8(R1), F8
       L.D F10, -16(R1)
       ADD.D F12, F10, F2
       S.D -16(R1), F12
       L.D F14, -24(R1)
       ADD.D F16, F14, F2
       S.D -24(R1), F16
       DSUBUI R1, R1, #32
       BNEZ R1, Loop;  Assume delayed branch
```

Rename registers

Remove some branch overhead (calculate intermediate values)

Loop Unrolling

Scheduling the loop for simple pipeline

```
Loop:  L.D F0, 0(R1)
       L.D F6, -8(R1)
       L.D F10, -16(R1)
       L.D F14, -24(R1)
       ADD.D F4, F0, F2
       ADD.D F8, F6, F2
       ADD.D F12, F10, F2
       ADD.D F16, F14, F2
       S.D 0(R1), F4
       S.D -8(R1), F8
       S.D -16(R1), F12
       DSUBUI R1, R1, #32
       BNEZ R1, Loop ; Assume delayed branch
       S.D 8(R1), F16
```

How to schedule for multiple issue?

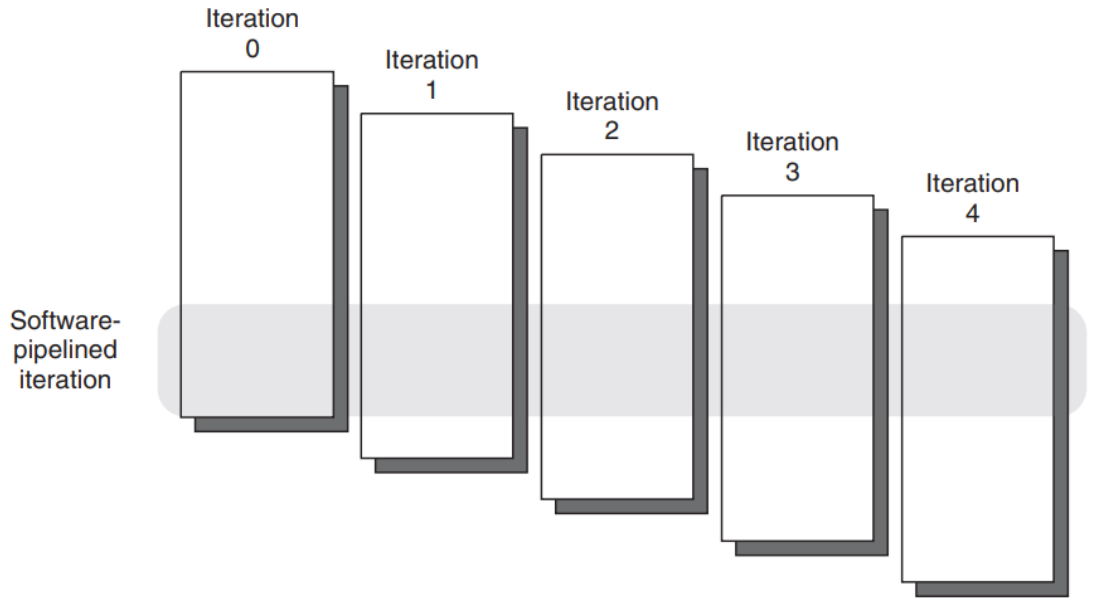
Software Pipelining (Section H.3)

Pipeline loops in software

Pipelined loop iteration

- Executes instructions from multiple iterations of original loop
- Separates dependent instructions

Less code than unrolling



Software Pipelining – Example

```
sum = 0.0;
for (i=1; i<=N; i++) {      ; sum = sum + a[i]*b[i]
   load a[i]              ; Ai
   load b[i]              ; Bi
   mult ab[i]             ; *i
   add sum[i]             ; +i
}
```

```
sum = 0.0;
START-UP-BLOCK
for (i=3; i<=N; i++) {
   load a[i]              ; Ai
   load b[i]              ; Bi
   mult ab[i-1]           ; *i-1
   add sum[i-2]           ; +i-2
}
FINISH-UP-BLOCK
```

```
                        LOOP
START-UP     i=3 ...     i=N      FINISH-UP
--------     ---         ---      ---------
A1    A2     A3   Ai     AN
B1    B2     B3   Bi     BN
       *1     *2   *i-1   *N-1     *N
              +1   +i-2   +N-2     +N-1    +N
```

Global Scheduling

Loop unrolling and software pipelining work well for straightline code

What if code has branches?

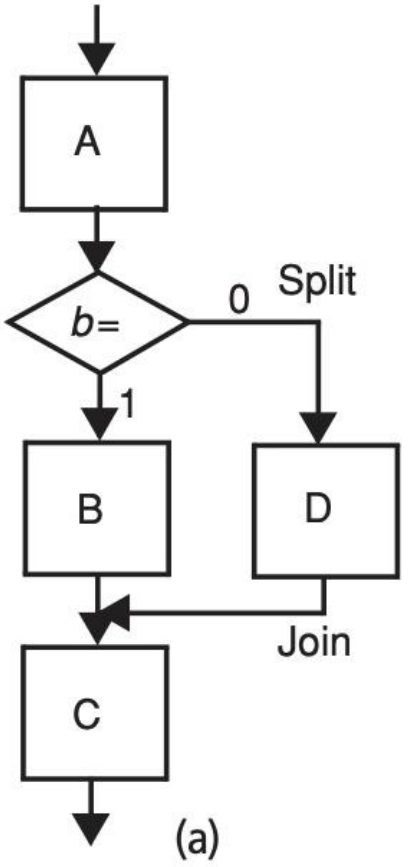
Global scheduling techniques

- Trace scheduling

Trace Scheduling

Compiler predicts most frequently executed execution path (trace)

Schedules this path and inserts repair code for mispredictions



Trace Scheduling - Example

```
b[i] = ``old''
a[i] =
if (a[i] == 0) then
    b[i] = ``new''; common case
else
    X
endif
c[i] =
```

Until done

- Select most common path - a trace
- Schedule trace across basic blocks
- Repair other paths

trace to be scheduled:

```
b[i] = ``old''
a[i] =
b[i] = ``new''
c[i] =
if (a[i] != 0) goto A

B:
```

repair code:

```
A: restore old b[i]
    X
   maybe recalculate c[i]
    goto B
```

Hardware Support to Expose Compile-Time ILP

Compiler scheduling limited by knowledge of branch behavior

Hardware support to help compiler

- Predicated (or guarded or conditional) instructions
- Hardware support for compiler speculation

Predicated Instructions (Section H.4)

Used to convert control dependence to data dependence

Instruction executed based on a predicate (or guard or condition)

If condition is false, then no result write or exceptions

Predicated Instructions (Cont.)

Example

```
if (condition) then {
    A = B;
}
...
```

Convert to:

`R1` ← result of condition evaluation

A = *B* predicated on `R1`

...

Hardware can schedule instructions across the branch

Alpha, MIPS, PowerPC, SPARC V9, x86 (Pentium) have conditional moves

IA-64 has general predication - 64 1-bit predicate bits

Limitations

- Takes a clock even if annulled

Hardware Support for Compiler Speculation (Section H.5)

Successful compiler scheduling requires

- Preservation of exception behavior on speculation
- Mechanism to speculatively reorder memory operations

Hardware for Preserving Exception Behavior

What if there is an exception on a speculative instruction?

Distinguish between two classes of exceptions

(1)Indicate program error and require termination (e.g., protection violation)

(2) Can be handled and program resumed (e.g., page fault)

Type (2) can be handled immediately even for speculative instructions

Type (1) requires more support

Poison bits

Poison Bits

Hardware support

- A poison bit for each register
- A speculation bit for each instruction

If a speculative instruction sees an exception

- it sets poison bit of destination

If a speculative instruction sees poison bit set for source

- it propagates poison bit to its destination

If normal instruction sees poison bit for source, takes exception

Normal instruction resets poison bit of destination register

Hardware for Memory Speculation

How to reorder memory ops if compiler is not sure of addresses?

Consider moving a load

- Insert a special check instruction at original location of load
- When load is executed, hardware saves its address
- If there is a store to L's address before the check instruction
  - Redo load
  - Branch to fix up code if other instructions already used load's value

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Qianni Deng (SJTU)
- Changsheng Xie (HUST)